

orchestrator/ 代码说明

给第一次看这份代码的人快速建立地图用的开发文档。`main.py` 只是薄薄的入口胶水代码，真正的 5 个 Agent 各自一个文件放在 `agents/` 目录下，`persona / store / trip_plan` 三个模块已经被对应的 Agent 接起来了（具体接了什么见文末），只有 OTA/酒店 Agent 还是纯占位假数据。

设计背景/接口契约见 [agent-architecture.md](#)（为什么这么分 Agent、为什么中心化编排）和 [agent-interfaces.md](#)（字段契约、原本的 Dify 方案，后来改成直接写 Python）。

环境准备

```
pip install openai python-dotenv chromadb
```

在 `orchestrator/.env` 里写一行（这个文件已经在 `.gitignore` 里，不会被提交）：

```
PARATERA_API_KEY=你的key
```

每个文件都能直接 `python orchestrator/<路径>.py` 单独跑（包括 `agents/` 里的 5 个文件），文件末尾的 `if __name__ == "__main__":` 都是自测代码，可以照着抄用法。

目录结构

```
orchestrator/
  llm_tool.py           # 通用 LLM 调用工具，跟"编排"本身无关，谁都能 import
  agents/
    orchestrator_agent.py # 编排 Agent: new_shared_state + classify_intent + orchestrate
    content_agent.py      # 达人/内容 Agent
    route_agent.py        # 行程/路线 Agent
    ota_hotel_agent.py     # OTA/酒店 Agent（还是纯占位）
    exception_agent.py    # 异常应变 Agent
  main.py               # 薄入口: re-export 编排 Agent 的两个函数 + __main__ 完整 demo
```

```
store.py / trip_plan.py / persona.py  # 不变
server.py / web/                      # 不变，还是 import main，接口不受这次拆分影响
```

拆分原则：谁负责哪个 Agent 就改 `agents/` 下自己那一个文件，不用碰 `main.py` 或别人的 Agent 文件，减少多人协作冲突。

llm_tool.py -- 通用 LLM 工具

职责：跟"编排"这件事本身无关的基础设施，5 个 Agent 里任何一个要调用模型都用这个，不要各自再写一遍 client 初始化。

函数/常量	作用
<code>call_llm(messages, model=MODEL_FULL)</code>	统一的模型调用入口
<code>MODEL_FULL / MODEL_LIGHT</code>	<code>DeepSeek-V4-Pro / DeepSeek-V4-Flash</code> ，两档模型对齐 <code>agent-architecture.md</code> 的模型档位设计

agents/orchestrator_agent.py -- 编排 Agent

职责：星型架构的中枢。接收用户消息 → 意图识别 → 决定调用哪几个子 Agent → 汇总结果 → 生成回复。对应 [agent-interfaces.md 第三节](#) 的节点拓扑，只是从 Dify 可视化节点换成了 Python 函数。模型档位用 `MODEL_FULL`（全系统推理最重）。

函数	作用
<code>new_shared_state(user_id, scenario="vacation", onboarding_answers=None)</code>	造一份共享状态： <code>persona</code> （真用 <code>persona.py</code> 的结构，传了 <code>onboarding_answers</code> 就走冷启动）+ <code>trip_plan</code> （真用 <code>trip_plan.py</code> 的结构）
<code>classify_intent(user_message)</code>	调模型判断这轮要触发 <code>content / route / booking / exception</code> 里的哪几个
<code>orchestrate(user_message, shared_state)</code>	主流程：意图识别 → 分发调用 4 个子 Agent（子 Agent 直接改 <code>shared_state</code> 里的 <code>trip_plan</code> ，不用再手动写回）→ 生成回复，返回 <code>(output, shared_state)</code>

agents/content_agent.py -- 达人/内容 Agent

职责：两阶段检索第一阶段。模型档位 `MODEL_FULL`（UGC 语义提炼），目前还没实际调用 LLM（纯向量检索）。

`run(shared_state, location_hint, top_k=3)`：算 `persona` 向量 → `store.query_similar_posts()` 查候选。
`location_hint` 留给"内容相关性排序"（两阶段检索第二阶段）用，那部分还没实现。

agents/route_agent.py -- 行程/路线 Agent

职责：把地点写进 `trip_plan` 某一天的行程节点。模型档位 `MODEL_LIGHT`，目前还没接 LLM，纯结构化写入。

`run(shared_state, places, time_budget=None)`：所有节点目前都写进同一个占位日期（`_PLACEHOLDER_DAY = "day-1"`），时间/交通方式字段是占位文字 "待定"。

agents/ota_hotel_agent.py -- OTA/酒店 Agent

职责：查库存/比价。模型档位 `MODEL_LIGHT`。**还是纯占位假数据**，没有真实的比价/查库存来源可接。真正确认预订后应该调 `trip_plan.add_hotel()` 落地，目前流程里没有"确认预订"这个触发点。

agents/exception_agent.py -- 异常应变 Agent

职责：按地点名字匹配 `trip_plan` 里的行程节点，命中就删掉并记一条天气异常。模型档位架构文档里写的是"中等模型"，目前只有 `MODEL_FULL` / `MODEL_LIGHT` 两档，先待定。

`run(shared_state, event_type, event_detail=None)`：`event_detail` 目前是纯子串匹配（整句用户消息去匹配地点名字），很粗糙，真实版本应该先做实体识别。

main.py -- 入口脚本

职责：纯胶水代码，不包含任何 Agent 逻辑。`from agents.orchestrator_agent import new_shared_state, orchestrate` 之后直接 re-export，`server.py` 还是 `import main` 调这两个函数，完全不用改。`__main__` 里是完整的端到端 demo（推荐 → 排路线 → 异常应变 → 打印最终行程）。

store.py -- 达人社区内容 + 历史记录

职责：达人社区的图文帖子存储 + 检索（两阶段检索第一阶段：按人格向量相似度筛选），以及行程反馈历史记录。

存储技术：Chroma 向量库（`orchestrator/data/chroma/`，本地文件持久化）存帖子；JSON Lines 文件（`orchestrator/data/history.jsonl`）存历史记录；图片本地落盘（`orchestrator/data/images/<post_id>/`），帖子里只存相对路径。

函数	作用
<code>save_image(post_id, filename, image_bytes)</code>	写一张图，返回存进 <code>content["images"]</code> 的相对路径
<code>resolve_image_path(相对路径)</code>	相对路径转本地绝对路径，读图用
<code>add_post(post_id, persona_vector, content)</code>	存一条帖子。 <code>content</code> 字段见下表
<code>query_similar_posts(persona_vector, top_k=5)</code>	两阶段检索第一阶段：按人格向量相似度找候选帖子，返回时带 <code>similarity_score</code>
<code>delete_post(post_id)</code>	删帖
<code>log_history(user_id, trip_id, record)</code>	追加一条行程反馈记录
<code>get_history(user_id=None)</code>	读历史记录，不传 <code>user_id</code> 读全部

`content` 字段（`add_post` 的第三个参数）：

字段	说明
<code>place / time_slot / avg_cost / rating / avoid_tips / verified_trip</code>	结构化字段，对齐 <code>agent-interfaces.md</code> 达人 Agent 输出契约
<code>caption</code>	帖子正文文字，可选
<code>images</code>	图片相对路径列表，可选，由 <code>save_image()</code> 生成

待定/限制：

- 内容相关性排序（两阶段检索第二阶段）没实现，留给调用方在 `query_similar_posts()` 的候选集里自己排
- 图片存本地文件系统，部署到 ModelScope Studio 时要换成对象存储（OSS/CDN），`images` 字段到时候存 URL 而不是本地路径，上层调用方式不变

trip_plan.py -- 旅游计划（行程/机票/酒店/天气异常）

职责：一次完整旅行的所有结构化信息，由多个 Agent 共同维护，最终汇总用于呈现。

顶层结构：

```
trip_plan = {
    "trip_id": "...",
    "flights": [...],          # OTA/酒店 Agent 维护
    "hotels": [...],          # OTA/酒店 Agent 维护
```

```
"weather_alerts": [...], # 异常应变 Agent 维护
"days": {日期: day_plan},# 行程 Agent 生成, 异常应变 Agent 增删改
}
```

景点通勤+餐饮的核心设计：每天一个"链表"，但不是指针式链表，而是**节点字典 + id 引用**（`day_plan["nodes"]` 是 `{node_id: node}`，每个节点有 `next_id` 指向下一个节点的 id，`day_plan["head_id"]` 是链表头）。这样设计是因为：

- 异常应变 Agent 改行程时只需要动被影响的那一个节点、接一下前后 `next_id`，不用重发整天的数组
- 本质是个普通 dict，直接能存 JSON/传 HTTP，不需要额外的链表↔数组转换逻辑

节点字段：`type`（`"attraction" | "meal"`）、`place`、`arrival_transport`、`arrival_time`、`end_time`、`next_id`；`meal` 类型节点多一个 `meal_type`（早餐/午餐/晚餐/夜宵）。

函数	作用
<code>new_trip_plan(trip_id)</code>	造一个空旅游计划
<code>add_flight / add_hotel / add_weather_alert</code>	往对应列表追加一条记录
<code>get_or_create_day(trip_plan, date)</code>	拿到某天的 <code>day_plan</code> ，不存在就新建
<code>add_stop(day_plan, node_id, type_, place, arrival_transport, arrival_time, end_time, after_id=None, **extra)</code>	插入一个节点， <code>after_id=None</code> 插到最前面，否则插到该节点后面
<code>remove_stop(day_plan, node_id)</code>	删除节点，自动重连前后节点（异常应变 Agent 常用，比如景点临时关闭）
<code>patch_stop(day_plan, node_id, **fields)</code>	只改某个节点的部分字段，不动链表结构
<code>day_stops(day_plan)</code>	把链表还原成有序数组，渲染/地图面板用
<code>render(trip_plan)</code>	汇总成最终展示结构 (<code>flights / hotels / weather_alerts / days</code> ，每天的 <code>stops</code> 已经是有序数组)

现状：已经接进 `main.py` 的 `shared_state["trip_plan"]`，`agent_route` 会往里面加节点、`agent_exception` 会删节点/记天气异常。目前所有节点都写进同一个占位日期 `"day-1"`（`main.py` 里的 `_PLACEHOLDER_DAY`），还没做真正的多日期规划。

persona.py -- 人格变量

职责：定义"旅游人格"的具体维度结构，供达人/行程/OTA Agent 读取，反馈闭环用来校准。

维度设计（能量化的维度都用了 0~1 连续分数，不是纯标签）：

维度	归属	类型	说明
taste	stable_traits (跨场景稳定)	固定词表 multi-hot	菜系标签, 词表见 _TASTE_VOCAB
novelty_score	stable_traits	float 0~1	0=稳妥大众, 1=小众冒险
pace_score	scenario_traits (按场景分开存)	float 0~1	0=完全休闲, 1=完全特种兵
budget_score	scenario_traits	float 0~1	0=穷游, 1=公务舱/总统套房/租车这种享受型消费; 硬过滤字段, 不进相似度向量 , 用 budget_filter_ok() 按数值区间过滤
social_mode	scenario_traits	分类 (solo / couple / family / friends)	并列模式非程度光谱, 向量里按 one-hot 编码
interest_theme	scenario_traits	固定词表 multi-hot	词表见 _INTEREST_VOCAB

scenario_traits 按"本次旅行模式" (business_trip / vacation / family / solo_adventure ...) 分开存, 同一个人不同场景下这些值可以完全不同。

函数	作用
new_persona(user_id)	造一份空人格
bootstrap_from_onboarding(user_id, scenario, answers)	冷启动: 从一份问卷答案组出初始 persona
set_scenario_traits(persona, scenario, **fields) / get_scenario_traits(persona, scenario)	读写某场景的特质, 写入时会校验词表 + clamp 数值到 0~1
update_stable_traits(persona, **fields)	读写跨场景稳定特质, 同样有校验
apply_feedback(persona, scenario, dimension, new_value, reason="")	按维度校准, 写入 + 留痕 (feedback_log)。"该改成什么值"由调用方决定, 这里只负责写入
budget_filter_ok(a_score, b_score, max_diff=0.25)	判断两个 budget_score 是否够接近, 硬过滤专用, 独立于相似度向量
compute_persona_vector(persona, scenario)	拼出向量: [novelty_score, taste multi-hot, pace_score, social_mode one-hot, interest_theme multi-hot], 固定 20 维 (词表大小变了维度也会变)

待定/限制：

- `compute_persona_vector()` 是数值特征拼接，不是真实语义 embedding，以后接真实 embedding 模型可以整体替换掉函数体，调用方（`store.py`）接口不用变
- 改 `_TASTE_VOCAB` / `_INTEREST_VOCAB` 词表会改变向量维度，如果本地 Chroma 已经有存量数据，改词表前要清掉 `orchestrator/data/chroma` 重新灌数据，不然新旧向量维度对不上（这个坑已经踩过一次）
- 从一条用户反馈文本判断"该把哪个维度调成什么值"的算法没实现，留给编排/达人 Agent 决定

server.py + web/index.html -- 呈现层（网页）

职责：把 `main.py` 的编排 Agent 包成 HTTP 接口，配一个两栏网页——左边实时日程，右边聊天框。原本 `agent-architecture.md` 里设计的是三面板（社区/地图/聊天），这版先简化成两栏，验证"聊天真的能驱动日程展示"这条链路，社区面板/地图面板以后再加。

运行方式：

```
pip install flask
python orchestrator/server.py
```

然后浏览器打开 `http://127.0.0.1:5000`。

接口（都是同一个 Flask app 提供，同源不用处理 CORS）：

接口	作用
GET /	返回 <code>web/index.html</code>
GET /trip	返回 <code>trip_plan.render()</code> 的当前完整行程，左侧日程面板用这个渲染
POST /chat	body 传 <code>{"message": "..."} </code> ，内部调 <code>main.orchestrate()</code> ，返回编排 Agent 的输出（ <code>chat_reply</code> / <code>community_panel</code> / <code>map_panel</code> ）

前端逻辑（`web/index.html`，原生 HTML/CSS/JS，没引入任何框架）：发消息 → 调 `/chat` 显示回复 → 再调一次 `/trip` 刷新左侧日程，这样每次对话后日程面板都是最新状态。

现状/限制：

- 全局只有一个进程内共享状态（`server.py` 里的 `_state`），是单会话 demo，没有登录/多用户/并发处理，仅供本地演示，不要直接这样部署到公网
- 已经端到端测试过："推荐+排路线"这类消息发过去，`/trip` 能看到新加的行程节点；异常应变消息发过去，`/trip` 能看到节点被删掉——聊天确实驱动了日程展示
- 社区图文面板、地图面板（三面板设计里剩下的两块）还没做进网页

待接事项

四个文件已经在 `main.py` 里串起来了（`persona / store / trip_plan` 都接了），跑 `python orchestrator/main.py` 能看到一次完整的"推荐→规划路线→异常应变→重新渲染行程"的端到端流程。还剩这几处没做：

1. `agent_ota_hotel` 完全没接，还是纯占位假数据，没有真实比价/查库存来源可接
2. `agent_route` 目前所有节点都写进同一个占位日期 `_PLACEHOLDER_DAY = "day-1"`，没有真正的多日期规划；交通方式/到达时间也都是占位文字 "待定"
3. `agent_exception` 用整句用户消息去子串匹配地点名字，很粗糙（比如"西湖"两个字出现在消息里就命中），真实版本应该先做实体识别抽出具体地点
4. 两阶段检索第二阶段（候选集内部按内容相关性排序）还没实现，`agent_content` 里 `location_hint` 参数目前没用上
5. 反馈闭环（用户点评行程 → 校准 `persona`）还没接：`store.log_history()` 记录和 `persona.apply_feedback()` 校准都写好了，但没人在 `orchestrate()` 里调用它们